

Lab 4 - VibeCoding

Clean Chat Transcript - Fruit Ninja Project

Repository: [spoorthy0805/41_fruit-ninja](#)

This transcript keeps the substantive prompts, implementation guidance, testing, and submission steps. Screenshots and repetitive acknowledgements are omitted for readability.

1. Setup and baseline recording

User

https://github.com/SETAPESU26/41_fruit-ninja
ok tell me what to do STEP BY STEP

ChatGPT

Follow the lab in this order: fork the assigned repository, clone your fork, install requirements, run the original game, record the before video, complete each README task one by one with separate commits, record the final after video, and push only to your own fork. Do not raise a pull request to the SETAPESU26 repository.

User

I have created the fork and opened it in VS Code. What next?

ChatGPT

Open a VS Code terminal and verify Python, then install the dependencies and run the original game. Do not edit the code before making the baseline recording.

```
python --version
pip install -r requirements.txt
python main.py
```

User

Is this before recording fine?

ChatGPT

Yes. It clearly shows the original game running before modifications, so it is suitable as the before video.

2. Task 1 - Fast swipe collision detection

User

Here is my current game_engine.py. The game only checks fruit.contains_point(x, y) during mouse motion.

ChatGPT

The bug is that only the current mouse point is tested. A fast swipe can jump from one side of a fruit to the other between events. The fix is to test the entire line segment from the previous mouse position to the current position against the fruit circle.

ChatGPT - Task 1 implementation

The collision fix adds a segment-to-circle test and uses the previous trail point as the start of the swipe. The existing fruit.py can remain unchanged.

```
def _segment_hits_circle(self, start, end, fruit):
    x1, y1 = start
    x2, y2 = end

    dx = x2 - x1
    dy = y2 - y1

    if dx == 0 and dy == 0:
        return fruit.contains_point(x1, y1)

    length_squared = dx * dx + dy * dy
    t = ((fruit.x - x1) * dx + (fruit.y - y1) * dy) / length_squared
    t = max(0, min(1, t))
```

```

        closest_x = x1 + t * dx
        closest_y = y1 + t * dy

        distance_squared = (
            (fruit.x - closest_x) ** 2 +
            (fruit.y - closest_y) ** 2
        )

        return distance_squared <= fruit.radius ** 2

def _handle_motion(self, pos):
    previous_pos = self.trail[-1] if self.trail else pos

    for fruit in self.fruits:
        if not fruit.sliced and self._segment_hits_circle(previous_pos, pos, fruit):
            self._slice(fruit)

    self.trail.append(pos)

    if len(self.trail) > 15:
        self.trail.pop(0)

```

User

task 1 works

ChatGPT

Commit Task 1 separately.

```

git add game/game_engine.py
git commit -m "Improve fruit collision detection"

```

3. Task 2 - Proper Game Over screen

User

Here is my current main.py.

ChatGPT

Task 2 should stop normal gameplay when a bomb is sliced or lives reach zero, and display a proper Pygame Game Over overlay with the final score instead of only printing to the terminal.

ChatGPT - Task 2 implementation

The game engine was updated with separate fonts for the game-over title and final score, and a `render_game_over()` method. The render method calls it whenever `self.game_over` is true.

```

def render_game_over(self, screen):
    overlay = pygame.Surface((self.width, self.height))
    overlay.set_alpha(180)
    overlay.fill((0, 0, 0))
    screen.blit(overlay, (0, 0))

    game_over_text = self.game_over_font.render("GAME OVER", True, WHITE)
    score_text = self.final_score_font.render(
        f"Final Score: {self.score}",
        True,
        WHITE
    )

    game_over_rect = game_over_text.get_rect(
        center=(self.width // 2, self.height // 2 - 40)
    )
    score_rect = score_text.get_rect(
        center=(self.width // 2, self.height // 2 + 35)
    )

```

```
screen.blit(game_over_text, game_over_rect)
screen.blit(score_text, score_rect)
```

User

task 2 works

ChatGPT

Commit Task 2 separately.

```
git add main.py game/game_engine.py
git commit -m "Add game over screen"
```

4. Task 3 - Replay and difficulty selection

User

Here is my current game_engine.py after Task 2.

ChatGPT

Task 3 adds replay choices to the Game Over screen: Easy, Medium, Hard, and Exit. Restarting resets score, lives, fruits, the trail, and the spawn timer. Each difficulty changes spawn interval, bomb chance, and speed scale.

ChatGPT - Difficulty settings used

```
Easy:
    spawn_interval = 70
    bomb_chance = 0.08
    speed_scale = 0.9

Medium:
    spawn_interval = 55
    bomb_chance = 0.15
    speed_scale = 1.0

Hard:
    spawn_interval = 40
    bomb_chance = 0.25
    speed_scale = 1.2
```

Keyboard options on the Game Over screen: 1 = Easy, 2 = Medium, 3 = Hard, 4 = Exit. The main loop checks engine.exit_requested and closes the window when Exit is selected.

User

ok done

ChatGPT

Task 3 is complete. Commit it separately before moving to Task 4.

```
git add main.py game/game_engine.py
git commit -m "Add replay and difficulty selection"
```

5. Task 4 - Sound effects

User

What exactly does the README require for sound?

ChatGPT

The README requirement is: "Add basic sound effects for slicing a fruit, hitting a bomb, and the game-over moment." It does not require a particular audio source or file format.

User

What is the best approach for Task 4?

ChatGPT

Use Pygame's mixer and generate simple tones in code. This avoids external audio-file problems while still satisfying the README requirement. Use a high short tone for fruit slicing, a low buzz for a bomb, and a descending three-note sequence for Game Over.

ChatGPT - Sound generation code

```
def setup_sounds(self):
    try:
        pygame.mixer.init(frequency=44100, size=-16, channels=1)

        self.slice_sound = self.make_tone(900, 0.07, 0.35)
        self.bomb_sound = self.make_tone(100, 0.30, 0.65)

        self.game_over_sound = self.make_sequence([
            (500, 0.18),
            (350, 0.18),
            (200, 0.35)
        ], 0.5)

        self.bomb_channel = pygame.mixer.Channel(1)
        self.game_over_channel = pygame.mixer.Channel(2)

    except pygame.error:
        self.slice_sound = None
        self.bomb_sound = None
        self.game_over_sound = None

def make_tone(self, frequency, duration, volume):
    sample_rate = 44100
    sample_count = int(sample_rate * duration)
    buffer = array.array("h")

    for i in range(sample_count):
        value = math.sin(2 * math.pi * frequency * i / sample_rate)
        sample = int(32767 * volume * value)
        buffer.append(sample)

    return pygame.mixer.Sound(buffer=buffer)

def make_sequence(self, notes, volume):
    sample_rate = 44100
    buffer = array.array("h")

    for frequency, duration in notes:
        sample_count = int(sample_rate * duration)

        for i in range(sample_count):
            value = math.sin(2 * math.pi * frequency * i / sample_rate)
            sample = int(32767 * volume * value)
            buffer.append(sample)

    return pygame.mixer.Sound(buffer=buffer)

def _slice(self, fruit):
    fruit.sliced = True

    if fruit.kind == "bomb":
        if self.bomb_sound and self.bomb_channel:
            self.bomb_channel.play(self.bomb_sound)
```

```
        if self.game_over_sound:
            self.bomb_channel.queue(self.game_over_sound)

        self.game_over_sound_played = True
        self.game_over = True

    else:
        if self.slice_sound:
            self.slice_sound.play()

        self.score += 1
```

User

I tested it. The sounds are distinct and the final recording looks fine.

ChatGPT

Task 4 is complete. Commit it separately.

```
git add game/game_engine.py
git commit -m "Add game sound effects"
```

6. Final recording and submission

User

Is my final video good enough?

ChatGPT

Yes. The shorter final recording is suitable as the after video. It shows the completed game and is close to the requested 10-second duration.

User

I am done committing. What next?

ChatGPT

Use the same forked repository; do not create a new repo. Create a Lab-4 folder in the existing project. Put the original before recording, the final after recording, and this chat-history PDF in that folder.

ChatGPT - Final folder structure

```
Lab-4/
  before.mp4
  after.mp4
  chat-history.pdf
```

Then commit the deliverables and push the main branch to your personal fork. Do not open a pull request to SETAPESU26.

```
git add Lab-4
git commit -m "Add Lab 4 deliverables"
git push origin main
```